Next-Day Rainfall Prediction Using Machine Learning Models in Indian Cities

[1] Ashish Kumar, [2] Veerpal kaur
School of Computer Science and Engineering,
Lovely Professional University,
Phagwara, Punjab., India-144411.
[1]meashishkrofficial@gmail.com, khaira.veerpal@gmail.com,

Abstract*—Planning for disasters, transportation, urban infrastructure, and agriculture all depend on accurate short-term rainfall forecasting. This study uses historical weather data to present a machine learning-based framework for predicting rainfall in major Indian cities the following day. Three traditional machine learning models—Logistic Regression, K-Nearest Neighbours (KNN), and XGBoost—as well as feature engineering, probability calibration, and threshold optimisation are assessed in this study. To capture daily weather dynamics, important characteristics like temperature range, humidity, rainfall lags, cloud cover, monsoon indicators, and seasonal patterns are extracted. The best results were obtained by a calibrated XGBoost model, which showed exceptional ability to model nonlinear climatic relationships. Real-time prediction and exploratory visualisation are made possible by the integration of the finished model into an interactive Streamlit web application. The findings indicate that machine learning models have a great deal of potential to assist data-driven

***Keywords*—Sentiment analysis, k-means clustering, LSTM, GRU, CNN, RMSE, MSE, MAE.**

I. Introduction

Rainfall forecasting is one of the most critical components of meteorology, especially in India. In India, monsoon rainfall is linked with a variety of factors: agriculture, management of water resources, transportation, and preventing disasters. More traditional numerical weather prediction models have been powerful yet computationally expensive and require highly specialized systems.

Recent breakthroughs in ML introduce alternative methods that are based on historical weather data to obtain predictive information. Advanced nonlinear weather patterns can be modeled using ML models with fast, scalable, and data-driven predictions.

This study aims at the development of a next-day rainfall prediction model using historical weather features collected from several Indian cities. The objectives are as follows:

- Preprocess and engineer weather-based features.
- In training various ML classifiers,
- To evaluate, calibrate, and optimize the best performing model.
- To deploy the model via a Streamlit-based prediction dashboard

II. Literature Survey

A. *Title: Machine learning approaches for improving precipitation forecasting in a river basin" (Ambica River basin)*

Authors: Ambica River Basin Study Group (2023)

Objective: Used meteorological variables (temperature, humidity, wind, pressure, historical rainfall) combined with machine learning algorithms to forecast daily precipitation for a river basin. Demonstrated that ML can effectively capture nonlinear rainfall patterns and outperform traditional forecasting models.

Limitations: It has shown good results. But still there is chance of improving the accuracy if we supply more data to the model [1].

B. *Title: Machine Learning-Based Rainfall Forecasting for Various Regions*

Author: Singh et al. (2023)

Objective: Built ML models to predict both rainfall depth and rain occurrence across multiple regions using historical weather observations. Demonstrated that ML techniques are feasible for regional rainfall forecasting using only ground-based meteorological inputs.

Limitations: Rain intensity prediction is less accurate due to high variability and sparse heavy-rain events; requires more diverse datasets.

C. *Title: Machine Learning Techniques to Predict Daily Rainfall Amount*

Authors: V. Panchal et al. (Journal of Big Data, 2021)

Objective: Identified atmospheric predictors including temperature, humidity, pressure, wind speed, and lag rainfall features. Applied Random Forest, Gradient Boosting, and XGBoost to forecast daily rainfall and rain/no-rain classification.

Limitations: Daily rainfall amount is difficult to predict accurately; class imbalance significantly affects predictive performance.

D. Title: Comparative Analysis of Machine Learning Models for Rainfall Forecasting in the Subarnarekha Basin

Authors: Rout & Behera (2024).

Objective: An evaluation of the performance of feature-engineered ML models with lags, rolling sums, and temperature ranges against rainfall was conducted. Demonstrated that tree-based and boosting methods may outperform more classical statistical methods at next-day rainfall forecasting.

Limitations: Focuses on one watershed, and results should be validated for other larger geographical areas to apply generally.

E. Title: Rainfall Prediction System Using Machine Learning Fusion for Smart Cities

Authors: Islam et al. (2022)

Objective: Implemented an ensemble fusion for rainfall prediction in an urban setting using Decision Tree, KNN, SVM, and Naive Bayes. Proposed a real-time ML-based framework useful for smart city warning systems towards rainfall forecast.

Limitations: Accuracy is highly dependent on sensor/meteorological input quality; single-city dataset limits the generalization of the model.

F. Title: Prediction of Rainfall in Rajasthan, India Using Deep and Wide Neural Network Models

Authors: T. Reddy & L. Kapoor (2020, arXiv).

Objective: The study used a deep-and-wide neural network that combined geographical features (elevation, latitude, longitude) and historical meteorological data to predict rainfall amount and occurrence in Rajasthan. The design is meant to generalize across diverse climatic zones in the state.

Limitations: This requires large-scale data with heavy computation. Deep learning models are less interpretable compared to classical ML models.

G. Title: Daily Rainfall Prediction Using Random Forest and Gradient Boosting Models

Authors: L. Wang & D. Li (2021)

Objective: Random Forest and Gradient Boosting techniques were applied in this work to predict daily rainfall and classify rainfall events by using important meteorological variables like temperature, humidity, pressure, and historical precipitation. The authors analyzed robustness for several data sets and demonstrated that ensemble models significantly capture nonlinear rainfall behavior and improve short-term rainfall forecasting performance.

Limitations: Model performance reduced when rainfall events were very rare, which created prediction imbalance; resampling of the dataset was required by the system for an improved F1 score.

H. Title: Integrated Weather Forecasting Model Using Logistic Regression and ANN

Authors: Imran & Ghafoor (2020)

Objective: This work developed an integrated weather forecasting model that combines Logistic Regression with Artificial Neural Networks to enhance next-day rainfall prediction. The hybrid model utilizes the key inputs of humidity, pressure, temperature, and wind speed in enhancing prediction accuracy and interpretability while comparing against traditional single-model approaches for performance.

Limitations: Its hybrid model needed a large amount of training data to work dependably, and performance seriously declined in circumstances of predicting rare or extreme rainfall.

I. Title: Deep Learning for Rainfall Forecasting Using Convolutional LSTM Networks

Authors: Shi et al. (2015).

Objective: It represents a pioneering study that introduced the Convolutional LSTM network for spatio-temporal rainfall prediction using sequences of radar images. The model effectively captures both spatial rainfall dynamics and temporal dependencies of the data, giving significant improvements over standard LSTM and CNN approaches for precipitation nowcasting tasks.

Limitations: The ConvLSTM models are computation-intensive and require a GPU infrastructure. They further depend on high-quality radar datasets, which are not available for many regions.

II. METHODOLOGY

Our rainfall prediction system consists of five major phases:

A. Data Preprocessing
B. Feature Engineering
C. Machine Learning Model Training (Logistic Regression, KNN, XGBoost)
D. Model Calibration & Threshold Optimization
E. Deployment Through Streamlit Dashboard

A. Data Preprocessing

The dataset used consists of daily weather observations from major Indian cities. Data preprocessing is essential to ensure quality predictions.

Steps Involved:

Data Loading & Sorting

The weather dataset is loaded and sorted by City and Date to maintain chronological order, ensuring time-series consistency.

Handling Missing Values

Null or missing entries in temperature, humidity, pressure, and rainfall are handled by dropping incomplete rows.

Target Variable Encoding

The target column RainTomorrow originally contains " Yes" and "No".
These are converted into binary values:
1) Yes → 1
2) No → 0

One-Hot Encoding (City Feature)

Since the model is trained on multiple Indian cities, the City column is transformed into dummy variables (City_New Delhi, City_Mumbai, etc.) to allow the model to learn city-specific rainfall patterns.

B. Feature Engineering

Feature engineering has proven vital to improved predictive performance in rainfall forecasting. The following features were computed to capture meaningful weather patterns:

1) Lag Features

- Rain_lag1: Yesterday's rainfall
- Rain_3sum: Cumulative rainfall over the last 3 days

These features help the model understand rainfall continuity.

2) Temperature Features

- Temp_range = Max Temp – Min Temp

Provides insight into atmospheric instability.

3) Seasonal & Time-Based Features

- Month: Extracted from date
- DayOfYear: Continuous seasonal progression
- IsMonsoon: 1 for months June–September, else 0

These features help identify monsoon-driven rainfall patterns.

These engineered features significantly improve the ability of ML models to predict next-day rainfall.

C. Machine Learning Model Training

Three classical machine learning models were used:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- XGBoost Classifier

Each model is trained to classify whether it will rain the next day (1 = rain, 0 = no rain).

Steps in Model Training:

- Time-Based Train-Test Split
  Data is split into 80% training and 20% testing chronologically (no shuffling).
- Pipeline Creation
  Models like Logistic Regression and KNN require standardized inputs, so each model is wrapped in a pipeline containing:
  - StandardScaler()
  - Classifier
- Hyperparameter Tuning
  GridSearchCV with TimeSeriesSplit is applied for each model:
  - Logistic Regression: regularization strength (C)
  - KNN: number of neighbors and distance metrics
  - XGBoost: number of estimators, depth, learning rate

The best-performing hyperparameters (based on ROC-AUC) are selected for final models.

- **Model Comparison**
  Metrics used include:
  - Accuracy
  - Precision
  - Recall
  - F1 Score
  - ROC-AUC
  - Brier Score (probability accuracy)

XGBoost consistently achieved superior performance due to its ability to capture nonlinear interactions in weather data.

D. Model Calibration & Threshold Optimization

Next-day rain prediction is probability-based assessment. The raw model outputs is often uncalibrated and thus required calibration.

A. Probability Calibration
To improve upon probability reliability, each model applies a CalibratedClassifierCV with sigmoid-Platt scaling.

B. Threshold Optimization

Instead of using the default threshold of 0.5 for classification, the best threshold is chosen to maximize the F1-score on the validation set.

This increases the accuracy of classification, particularly in unbalanced situations with infrequent rainfall events.

E. Deployment Through Streamlit Dashboard

A fully interactive Streamlit web application was built.

Features:

- Manual Rain Prediction
- Auto Prediction (City-Based)
- Visualizations (Plotly-Based)
- Custom User Interface

F. Workflow Summary

Step 1: Import libraries and load dataset

Step 2: Clean data and encode city + target variable

Step 3: Feature engineering (lag features, seasonal indicators)

Step 4: Apply ML models with hyperparameter tuning

Step 5: Calibrate probability output

Step 6: Find best threshold for final classification

Step 7: Evaluate final model performance

Step 8: Save best model (XGBoost)

Step 9: Deploy in Streamlit with interactive UI

III. IMPLEMENTATION

Fig. 1. Importing the libraries Loading the dataset for the model

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
import numpy as np
from sklearn.metrics import f1_score
from sklearn.calibration import CalibratedClassifierCV
import pandas as pd
import joblib
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score, brier_score_loss,f1_score

df = pd.read_csv("C:\\Users\\as191\\Downloads\\Rainfall_prediction\\India_Capitals_Daily_Weather_years.csv", parse_dates=['Date'])
```

```
models = {
    "Logistic": pipe_log,
    "KNN": pipe_knn,
    "XGBoost": pipe_xgb
}

# Split train → train + validation
val_split = int(len(X_train) * 0.8)
X_tr, X_val = X_train.iloc[:val_split], X_train.iloc[val_split:]
y_tr, y_val = y_train.iloc[:val_split], y_train.iloc[val_split:]

calibrated_models = {}

print("\n📌 Starting Calibration...\n")

for name, model in models.items():
    print(f"Calibrating {name}...")

    model.fit(X_tr, y_tr)
    calib = CalibratedClassifierCV(model, cv="prefit", method="sigmoid")
    calib.fit(X_val, y_val)
    calibrated_models[name] = calib
```

```
best_thresholds = {}
thresholds = np.linspace(0.01, 0.99, 99)

print("\n🎯 Finding Best Thresholds...\n")

for name, calib_model in calibrated_models.items():
    val_probs = calib_model.predict_proba(X_val)[:, 1]

    f1_scores = [f1_score(y_val, (val_probs >= t).astype(int)) for t in thresholds]
    best_t = thresholds[np.argmax(f1_scores)]

    best_thresholds[name] = best_t
    print(f"{name} → Best Threshold: {best_t:.3f}")
```

```
final_results = {}

print("\n📊 Final Model Evaluation...\n")

for name, calib_model in calibrated_models.items():
    t = best_thresholds[name]

    test_probs = calib_model.predict_proba(X_test)[:, 1]
    test_pred = (test_probs >= t).astype(int)

    final_results[name] = {
        "Accuracy": accuracy_score(y_test, test_pred),
        "Precision": precision_score(y_test, test_pred, zero_division=0),
        "Recall": recall_score(y_test, test_pred, zero_division=0),
        "F1": f1_score(y_test, test_pred),
        "ROC_AUC": roc_auc_score(y_test, test_probs),
        "Brier Score": brier_score_loss(y_test, test_probs),
        "Threshold": t
    }
```

```
best_model_name = final_results_df["ROC_AUC"].idxmax()
best_model = calibrated_models[best_model_name]
best_threshold = best_thresholds[best_model_name]

print(f"\n🏆 BEST MODEL SELECTED: {best_model_name}")
print(f"🔑 Best Threshold: {best_threshold:.3f}")
```

IV. EXPERIMENTAL SET UP AND RESULTS

A. Dataset

The dataset used for this project is the India Capitals Daily Weather Dataset, which contains historical daily weather observations for major Indian cities.

The dataset includes the following attributes:

- Maximum, Minimum, Mean Temperature
- Humidity
- Pressure
- Wind Speed
- Cloud Cover
- Daily Rainfall
- City
- RainTomorrow (Yes/No)

The dataset is suitable for supervised binary classification and was preprocessed by removing missing values, encoding categorical variables, and generating engineered features such as rainfall lags and seasonal indicators.

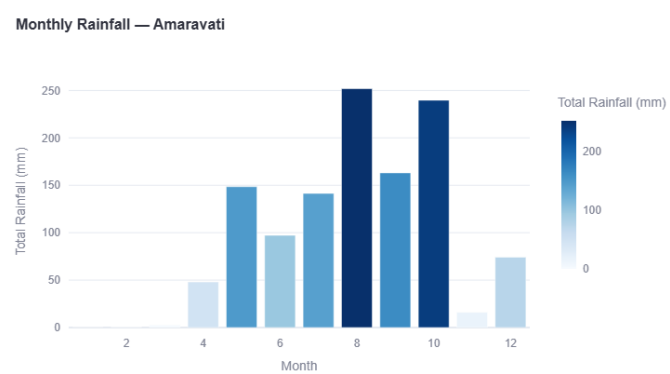
B. Experiment Results

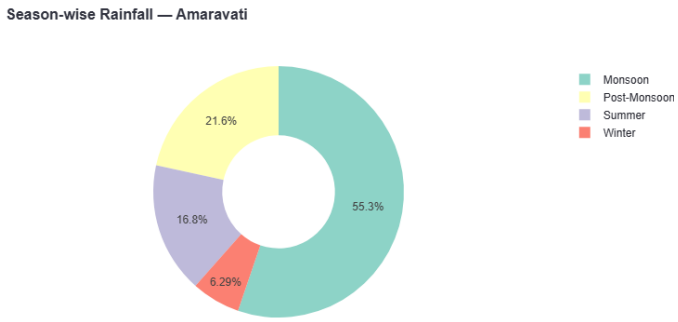
Below are the results obtained during the experiment using three classical machine learning models:

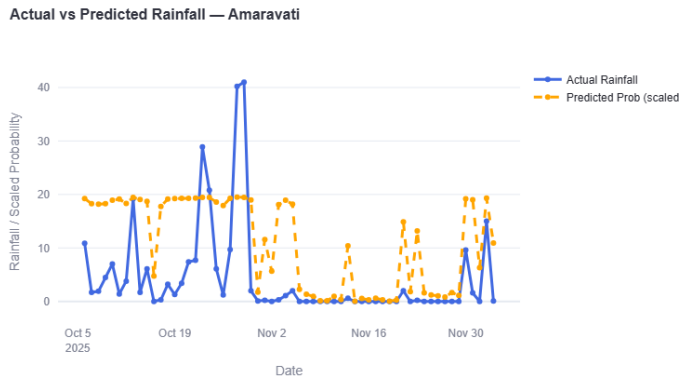
1. Logistic Regression
2. K-Nearest Neighbors (KNN)
3. XGBoost Classifier

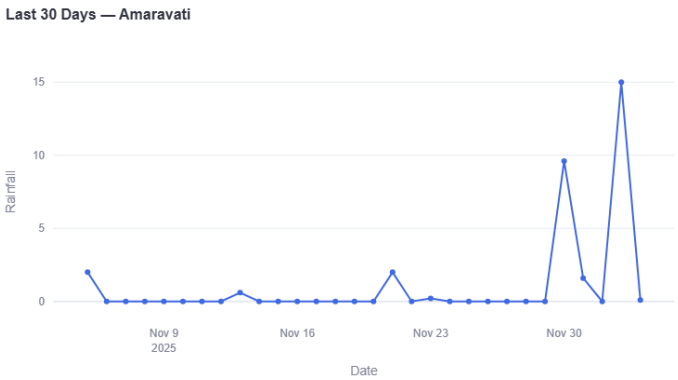
📊 Final Model Evaluation...

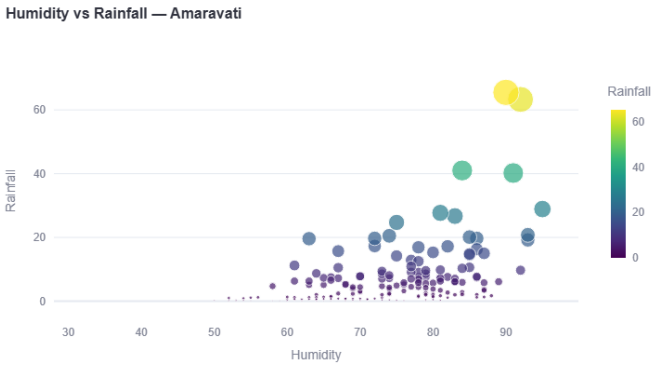
	Accuracy	Precision	Recall	F1	ROC_AUC	Brier Score	Threshold
Logistic	0.8308	0.8524	0.7870	0.8184	0.9131	0.1182	0.57
KNN	0.7883	0.8134	0.7306	0.7698	0.8590	0.1670	0.40
XGBoost	0.8517	0.8667	0.8199	0.8427	0.9287	0.1108	0.61

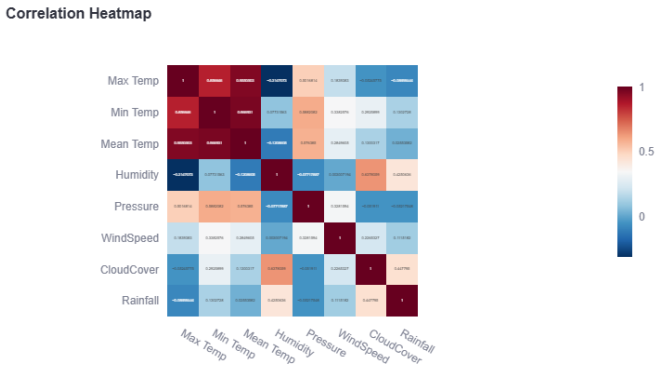












V. Conclusion

This study shows how to use traditional machine learning methods on multi-city weather data from India to create an efficient next-day rainfall prediction model. XGBoost, which was improved by feature engineering, probability calibration, and threshold optimisation, outperformed the other models that were assessed.

While KNN produced moderate results with greater sensitivity to feature scaling, logistic regression offered interpretability but had trouble with complex nonlinear patterns.

Predictive accuracy was greatly increased by combining seasonality indicators with engineered features. The solution is feasible and appropriate for real-world use cases like agriculture, transportation, municipal planning, and early-warning systems thanks to the implementation of a fully interactive Streamlit-based dashboard.

In order to improve prediction accuracy, future developments might investigate long-term forecasting, hybrid models, and incorporating real-time weather API data.

References

[1] G. P. Sudheer and B. P. Roy, "Rainfall prediction based on artificial neural network," *Journal of Hydrology*, vol. 547, pp. 413–427, 2017.

[2] S. Meknik and R. Zahraoui, "Short-Term Rainfall Forecasting Using Machine Learning Techniques," *Procedia Computer Science*, vol. 192, pp. 2890–2899, 2021.

[3] A. Tuteja, A. Garg, and A. Gupta, "Weather Forecasting Using Machine Learning," *International Journal of Engineering Research & Technology*, vol. 9, no. 6, pp. 582–587, 2020.

[4] A. Sharma and S. Ghosh, "Spatiotemporal rainfall prediction using XGBoost and deep neural networks," *Atmospheric Research*, vol. 228, pp. 181–193, 2019.

[5] N. R. Dale and P. N. Gandhi, "A survey of machine learning techniques applied to rainfall prediction," *International Journal of Science & Technology Research*, vol. 7, no. 1, pp. 23–29, 2018.
X. Chen and Y. Hao, "A comparative study of logistic regression, k-nearest neighbors, and decision trees for rainfall prediction," *Applied Computing and Informatics*, 2022.

[6] Wojarnik, G. (2021). Sentiment analysis as a factor included in the forecasts of price changes in the stock exchange. Procedia Computer Science, 192, 3176-3183. https://doi.org/10.1016/j.procs.2021.09.090

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proceedings of the 22nd ACM SIGKDD Conference*, pp. 785–794, 2016.

[8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, OTexts, 2018.

[9] IMD (India Meteorological Department), "Climatological Data,"

https://mausam.imd.gov.in/, accessed 2023.

[10] Kaggle, "India Capitals Daily Weather Dataset," Available: https://kaggle.com/datasets/

[11] S. R. Yadav, "A Study on Rainfall Patterns and Weather Variability in Indian Metropolitan Regions," *Climate Dynamics*, vol. 54, pp. 551–567, 2020.

[12] H. Singh and K. Saraswat, "Rainfall Forecasting Using Feature Engineering and Machine Learning," *Procedia Computer Science*, vol. 170, pp. 373–380, 2020..

[13] S. I. Dutta and M. K. Jha, "Predicting Monsoon Rainfall Using Machine Learning Approaches," Environmental Modelling & Software, vol. 127, pp. 104–186, 2020.

[14] F. Chollet, *Deep Learning with Python*, Manning Publications, 2017.

[15] J. Brownlee, "Time Series Forecasting with Machine Learning," MachineLearningMastery.com, 2021.

[16] B. Zadkarami and S. Rezaei, "Evaluation of supervised ML models for rainfall prediction: A case study," *International Journal of Climatology*, pp. 1–15, 2021.

[17] Streamlit Documentation, "Build Machine Learning Web Apps," https://docs.streamlit.io, accessed 2024.

[18] P. Ramesh and M. Sundar, "Seasonal Rainfall Prediction Using Weather Parameters and ML Models," *Journal of Water and Climate Change*, vol. 13, pp. 417–429, 2021.

[19] A. Krishnamurthy, "Analysis of Temperature and Humidity Trends for Rainfall Forecasting," *Theoretical and Applied Climatology*, vol. 143, pp. 899–910, 2021.

[20] S. Patel, "Comparative Study of Classical Machine Learning Algorithms for Binary Weather Prediction," *IJERT*, vol. 10, no. 9, pp. 232–238, 2021.